

PLP-Lab-Aula 10 - Material Complementar

EXERCÍCIO 1 - Exemplo: Soma de Vetores com CUDA (C++)

Soma de dois vetores.

RESPOSTA:

```
#include <iostream>

#include <cuda_runtime.h>

#define N 1024 // Tamanho dos vetores
#define THREADS_PER_BLOCK 256

// Kernel CUDA para somar dois vetores
__global__ void somaVetores(float *a, float *b, float *c, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n)
        c[i] = a[i] + b[i];
}

int main() {
    // Vetores na CPU (host)
    float *h_a, *h_b, *h_c;

    // Vetores na GPU (device)
    float *d_a, *d_b, *d_c;

    size_t size = N * sizeof(float);

    // Aloca memória na CPU
    h_a = (float*)malloc(size);
    h_b = (float*)malloc(size);
    h_c = (float*)malloc(size);
```

```

// Inicializa os vetores a e b
for (int i = 0; i < N; i++) {
    h_a[i] = i * 1.0f;
    h_b[i] = i * 2.0f;
}

// Aloca memória na GPU
cudaMalloc((void**)&d_a, size);
cudaMalloc((void**)&d_b, size);
cudaMalloc((void**)&d_c, size);

// Copia dados da CPU para GPU
cudaMemcpy(d_a, h_a, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_b, h_b, size, cudaMemcpyHostToDevice);

// Executa o kernel na GPU
int blocks = (N + THREADS_PER_BLOCK - 1) / THREADS_PER_BLOCK;
somaVetores<<<blocks, THREADS_PER_BLOCK>>>(d_a, d_b, d_c, N);

// Espera a GPU terminar
cudaDeviceSynchronize();

// Copia resultado de volta para CPU
cudaMemcpy(h_c, d_c, size, cudaMemcpyDeviceToHost);

// Exibe os primeiros 10 resultados
std::cout << "Soma dos vetores (primeiros 10 valores):\n";
for (int i = 0; i < 10; i++) {
    std::cout << h_a[i] << " + " << h_b[i] << " = " << h_c[i] << std::endl;
}

```

```

// Libera memória
cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);
free(h_a);
free(h_b);
free(h_c);

return 0;
}

```

EXERCÍCIO 2 - Multiplicação Escalar de Vetor com CUDA (C++)

Implemente um kernel CUDA que multiplica cada elemento de um vetor A por um escalar k, armazenando o resultado em um vetor B. Cada thread deve ser responsável por um único elemento.

- O vetor A possui $N = 256$ elementos.
- O escalar $k = 5$.
- Inicialize $A[i] = i$.
- Imprima os 10 primeiros elementos do vetor B.

RESPOSTA

```

#include <iostream>
#include <cuda_runtime.h>

__global__ void multiplicaEscalar(int *A, int *B, int k, int N) {
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < N) {
        B[i] = A[i] * k;
    }
}

```

```

int main() {

    const int N = 256;

    int *h_A = new int[N];

    int *h_B = new int[N];

    int *d_A, *d_B;

    int k = 5;


    // Inicializa o vetor A
    for (int i = 0; i < N; i++) {
        h_A[i] = i;
    }


    cudaMalloc(&d_A, N * sizeof(int));
    cudaMalloc(&d_B, N * sizeof(int));


    cudaMemcpy(d_A, h_A, N * sizeof(int), cudaMemcpyHostToDevice);


    // Configura 1 bloco de 256 threads
    multiplicaEscalar<<<1, 256>>>(d_A, d_B, k, N);


    cudaMemcpy(h_B, d_B, N * sizeof(int), cudaMemcpyDeviceToHost);


    // Imprime os primeiros 10 resultados
    for (int i = 0; i < 10; i++) {
        std::cout << "B[" << i << "] = " << h_B[i] << std::endl;
    }


    cudaFree(d_A);
    cudaFree(d_B);
    delete[] h_A;
}

```

```
delete[] h_B;  
  
return 0;  
}
```